Лабораторная работа 10. MLflow Registry и управляемый выпуск модели

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Продолжительность: 2 академических часа и самостоятельная работа Предшествующая работа: ЛР9 «Оркестрация обучения и обработка отказов»

1. Цель

Создать воспроизводимый процесс регистрации и выпуска модели: связать успешный Prefect run с MLflow model version, применить aliases `candidate` и `champion`, зафиксировать version tags и release manifest, отклонить модель при нарушении gate и выполнить rollback без переобучения.

2. Результаты обучения

Каждый студент должен уметь:

- различать MLflow run, logged model, registered model и model version;
- регистрировать sklearn-модель с signature и input example;
- загружать exact version и версию по alias;
- использовать aliases и tags вместо deprecated stages;
- объяснять автоматический gate и ручной approval;
- связывать commit, data hash, run ID и model version;
- доказать, что rejected candidate не изменяет champion;
- выполнить и объяснить rollback;
- диагностировать ошибку Registry по exact identifiers.

3. Командный результат

Команда предоставляет tag `lr10-v1` и:

1. локальный MLflow Tracking/Registry с database-backed backend;
2. registered model с уникальным именем команды;
3. минимум две model versions из разных run или параметров;
4. signature и безопасный input example у каждой принимаемой версии;
5. version tags происхождения и проверки;
6. aliases `candidate` и `champion`;
7. successful release manifest;
8. rejected gate report, не изменивший champion;
9. rollback manifest и подтверждение загрузки восстановленной версии;
10. тесты pure-логики manifest/gate и CI-проверку;
11. issue, небольшие PR, review и индивидуальные evidence cards.

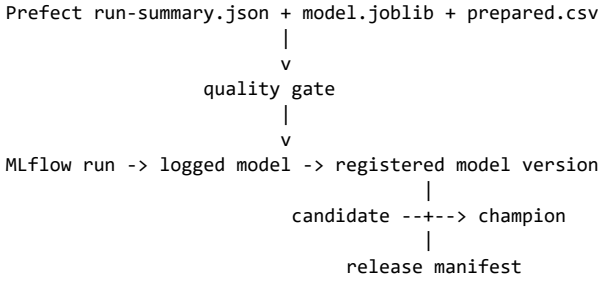
4. Проверенная версия и установка

На дату актуализации используется MLflow 3.16.x; зависимость проекта ограничена диапазоном `mlflow>=3.16,<4`. Перед семестром преподаватель повторно проверяет стабильную версию.

```
python -m pip install -e ".[dev,tracking,orchestration]"
mlflow --version
```

Stages `Staging`, `Production`, `Archived` и API `transition_model_version_stage()` не применяются. Используйте aliases и tags.

5. Локальная архитектура



Для Registry нужен database-backed backend. В учебном варианте используйте SQLite:

```
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlartifacts \
  --host 127.0.0.1 \
  --port 5000
```

В PowerShell обратный слеш замените обратным апострофом или выполните команду одной строкой. Файлы `mlflow.db`, `mlartifacts/` и `mlruns/` не добавляются в Git.

6. Именование

Registered model name:

```
<код-команды>-flight-delay-classifier
```

Пример: `team-03-flight-delay-classifier`. Имя не включает run ID, фамилию студента или слово `final`. Разные команды не используют общее имя.

7. Подготовка двух кандидатов

Выполните ЛР9 минимум дважды с разными безопасными параметрами, например `n_estimators=60` и `n_estimators=100`. Для каждого запуска сохраните собственный каталог и model path.

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr10-run-a \
  --model models/lr10-a.joblib \
  --n-estimators 60
```

Не копируйте один и тот же файл под двумя именами. В summary должны различаться параметры или подтверждённый источник обучения.

8. Регистрация candidate

Пример команды starter project:

```
python -m mlops_flight_delay.registry register \
  --tracking-uri http://127.0.0.1:5000 \
  --model-name team-03-flight-delay-classifier \
  --owner-team team-03 \
  --model models/lr10-a.joblib \
  --data reports/prefect/lr10-run-a/prepared.csv \
  --summary reports/prefect/lr10-run-a/run-summary.json \
  --source-commit <полный-Git-SHA> \
  --minimum-roc-auc 0.50 \
  --output reports/registry/lr10-candidate-a.json
```

Команда должна:

1. проверить `passed=true` и порог метрики в summary;
2. проверить hash подготовленных данных;
3. загрузить bundle и построить smoke prediction;
4. залогировать модель с signature и input example;
5. создать model version;

6. записать tags `validation_status`, `source_commit`, `data_sha256`, `owner_team`;
7. назначить alias `candidate`;
8. сохранить exact version и run ID в JSON.

9. Проверка Registry

В UI и через `MlflowClient` подтвердите:

- registered model name;
- version number;
- source run ID;
- signature;
- version tags;
- alias `candidate`;
- загрузку `models:/<name>/<version>`;
- загрузку `models:/<name>@candidate`;
- одинаковую resolved version для двух способов загрузки.

Скриншот допустим только как дополнение. Основное evidence — JSON, код и воспроизводимая команда.

10. Tags

Обязательные tags model version:

Key	Значение
`validation_status`	`passed` или `failed`
`source_commit`	полный Git SHA
`data_sha256`	hash prepared dataset
`quality_report`	относительный путь к report
`owner_team`	код команды
`release_status`	`candidate`, `approved` или `rolled_back`

Запрещены токены, пароли, персональные данные, полный dataset и локальные абсолютные пути.

11. Gate перед promotion

Candidate допускается к promotion, если одновременно:

- upstream summary имеет `passed=true`;
- test ROC AUC не ниже `--minimum-roc-auc`;
- data hash совпадает;
- model bundle загружается;
- signature создана;
- smoke prediction возвращает конечные вероятности в диапазоне `[0, 1]`;
- `source_commit` имеет формат полного SHA;
- version tags читаются обратно из Registry;
- review выполнен участником, не являющимся автором candidate.

Изменение порога фиксируется в issue и PR.

12. Promotion

```
python -m mlops_flight_delay.registry promote \
  --tracking-uri http://127.0.0.1:5000 \
  --model-name team-03-flight-delay-classifier \
  --data reports/prefect/lr10-run-a/prepared.csv \
  --approved-by student-reviewer-login \
  --output reports/releases/lr10-release.json
```

Promotion получает version по alias `candidate`, читает текущий `champion`, проверяет `validation_status=passed`, переназначает `champion` и затем читает alias повторно. Manifest должен сохранить `previous_champion_version`, даже если первого champion ещё не было.

13. Release manifest

Обязательные поля:

```
{
  "schema_version": "1.0.0",
  "model_name": "team-03-flight-delay-classifier",
  "model_version": "2",
  "previous_champion_version": "1",
  "mlflow_run_id": "...",
  "source_commit": "...",
  "data_sha256": "...",
  "resolved_model_uri": "models:/team-03-flight-delay-classifier/2",
  "quality_gate": {"passed": true, "minimum_roc_auc": 0.5},
  "decision": {"status": "approved", "approved_by": "..."}
}
```

Сведения берутся из Registry и отчётов, а не вводятся повторно вручную.

14. Rejected candidate

Создайте обратимый негативный сценарий: `--minimum-roc-auc 1.1` либо копию summary с `passed=false`. Ожидаемый результат:

- команда завершилась ненулевым exit code;
- создан failure report;
- новая версия не получила `champion`;
- текущий champion остался неизменным;
- причина содержит фактическую метрику и порог;
- повторный запуск с корректными входами возможен.

Нельзя имитировать отказ удалением общей базы, artifact store или версии другого участника.

15. Rollback

После успешного promotion выполните контролируемый откат на previous champion:

```
python -m mlops_flight_delay.registry rollback \
  --tracking-uri http://127.0.0.1:5000 \
  --model-name team-03-flight-delay-classifier \
  --target-version 1 \
  --data reports/prefect/lr10-run-a/prepared.csv \
  --reason "controlled-lr10-drill" \
  --approved-by student-operator-login \
  --output reports/releases/lr10-rollback.json
```

После alias update загрузите `models:/<name>@champion`, выполните smoke prediction и запишите resolved version. Отозванная version не удаляется.

16. Идемпотентность и конкуренция

Повторное чтение и проверка не должны создавать version. `register` создаёт новую version только для нового принятого артефакта. `promote` не создаёт version и не переобучает модель.

Для учебной работы promotion выполняется последовательно. Перед update команда повторно получает candidate и сравнивает его с reviewed version. Если alias изменился, операция прекращается и требует

нового review.

17. Тестирование

Обязательные тесты:

1. manifest содержит все обязательные поля;
2. неверный commit SHA отклоняется;
3. `passed=false` отклоняется;
4. метрика ниже порога отклоняется;
5. hash несоответствие отклоняется;
6. promotion требует `validation_status=passed`;
7. previous champion сохраняется;
8. rollback требует существующую target version;
9. секретоподобные ключи не попадают в manifest;
10. integration smoke создаёт version в временной SQLite Registry.

Pure tests должны работать без поднятого HTTP server. Integration test можно маркировать отдельно.

18. CI

CI устанавливает `[dev,tracking,orchestration]`, запускает pure tests и локальный registry smoke с отдельной временной SQLite database. Долговременные `mlflow.db` и artifacts не публикуются; в diagnostic artifact сохраняются только sanitized JSON reports.

19. Командные роли

Для команды 2–5 человек роли совмещаются или распределяются:

- владелец регистрации и signature;
- владелец gate и отрицательного сценария;
- владелец tags и manifest;
- release operator и rollback;
- reviewer происхождения и CI.

Роли должны отличаться от ЛР9. Координатор без технического или проверочного артефакта не считается отдельным вкладом.

20. Индивидуальная сдача участника команды

Командный результат: tag `lr10-v1`, registered model, versions, aliases, manifests и reports.

Каждый участник размещает отдельную evidence card. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР10 и tag `lr10-v1`
Роль	роль в ЛР10 и отличие от ЛР9
Задача	issue с наблюдаемым критерием
Личный артефакт	PR, test, version, tag policy, manifest, review или rollback report
Exact identifiers	commit SHA, run ID, model name и version
Решение	почему выбран alias/tag/gate и рассмотренная альтернатива
Риск	stale candidate, неверный hash, broken artifact, секрет или конфликт promotion

Поле	Что указать
Отказ	личный rejected/promotion/rollback сценарий и ожидаемое состояние champion
Использование ИИ	инструмент, самостоятельно проверенные утверждения или отсутствие использования

Минимальный допуск

1. Командная версия зафиксирована неизменяемым tag.
2. Участнику назначена собственная issue.
3. Есть содержательный личный артефакт; число commit не является доказательством.
4. Изменение прошло PR и review другого участника.
5. Evidence содержит рабочие exact identifiers и ссылки.
6. Автоматические проверки зелёные либо исключение письменно принято преподавателем.
7. Участник не является одновременно единственным автором и единственным approving reviewer своего promotion.

21. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag `lr10-v1`;
2. называет model name, exact version, run ID и commit;
3. загружает exact version или версию по alias;
4. показывает signature, tags и report;
5. объясняет различие `candidate` и `champion`;
6. воспроизводит назначенный rejected или rollback-сценарий;
7. доказывает, что champion изменился или остался прежним согласно сценарию;
8. объясняет один риск и способ обнаружения.

Преподаватель может изменить порог, candidate alias, target rollback или вход smoke prediction.

22. Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, новый report и обновлённую evidence card. Первая попытка не удаляется.

Стоп-факторы:

- использование deprecated stages как основного механизма;
- отсутствие signature или source run;
- alias назначен версии с failed gate;
- manifest заполнен вручную и противоречит Registry;
- champion переназначен без review/approval;
- rollback переобучает модель или удаляет историю;
- один общий failure-сценарий выдан всем участникам;
- в репозитории оказались токены, база с чужими данными или персональные сведения;
- скриншот заменяет код, exact identifiers и reports.

23. Критерии оценивания

Критерий	Баллы	Полное выполнение
Registration и signature	2	две versions связаны с run и загружаются
Gate и происхождение	2	commit, data hash, метрики и evidence проверяются
Aliases, tags и promotion	2	candidate/champion применены без stages
Manifest и rollback	2	previous champion сохранён и откат воспроизводим
Индивидуальная защита	2	личный артефакт, отказ, диагностика и решение

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

24. Чек-лист команды

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ MLflow 3 используется из проверенного диапазона.
- ☐ Backend Registry основан на SQLite или другой database-backed store.
- ☐ Зарегистрированы минимум две реальные model versions.
- ☐ У model versions есть signature и source run.
- ☐ Aliases `candidate` и `champion` разрешаются в exact versions.
- ☐ Tags происхождения читаются обратно из Registry.
- ☐ Rejected candidate не изменяет champion.
- ☐ Release manifest сформирован из фактических данных.
- ☐ Rollback возвращает предыдущую version без retraining.
- ☐ Smoke prediction выполнен после promotion и rollback.
- ☐ CI и тесты успешны.
- ☐ У каждого участника собственная evidence card и failure-сценарий.
- ☐ Итоговая версия отмечена tag `lr10-v1`.

25. Связь с ЛР11

В ЛР11 сервис будет публиковать resolved model name/version в структурированном логе и метриках. Наблюдаемость должна позволять сравнить поведение разных выпусков и связать деградацию с release manifest.

26. Официальные источники

- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/
- MLflow Registry tutorial: https://mlflow.org/docs/latest/ml/model-registry/tutorial
- MLflow Client API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.client.html
- MLflow package releases: https://pypi.org/project/mlflow/